

Paso 6 - Repetir con `while` | Ejercicios

`while` para cuando **no sabes** cuántas vueltas vas a dar: validar, insistir, un menú. Si alguno se te queda pegado, **Ctrl + C**.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B6_1_Centinel.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B6.1 - La suma con centinela

Qué practicas: el patrón centinela (leer antes del `while` y otra vez al final del cuerpo), y evitar una división por cero.

Pide números enteros hasta que el usuario escriba `0`. Al final muestra la suma, cuántos números ingresó y el promedio.

```
Numero (0 para terminar): 10
Numero (0 para terminar): 25
Numero (0 para terminar): 7
Numero (0 para terminar): 0
Ingresaste 3 numeros
Suma: 42
Promedio: 14.0
```

Ojo con el caso en que el primer número es `0`: no puede dividir por cero al sacar el promedio. Pruébalo y arréglalo.

B6.2 - Adivina el número

Qué practicas: un `while` cuya condición depende de una comparación, y generar un número al azar con `Math.random`.

El programa tiene un número secreto (parte con `int secreto = 7;`) y el usuario tiene que adivinarlo. En cada intento dice `Mas alto` o `Mas bajo`, y al final cuántos intentos tomó.

```
Adivina (1 a 10): 3
Mas alto
Adivina (1 a 10): 9
Mas bajo
Adivina (1 a 10): 7
Acertaste en 3 intentos!
```

Extras:

1. Que el secreto sea al azar: `int secreto = (int) (Math.random() * 10) + 1;`
2. Que solo tenga 3 intentos, y si se le acaban muestre `Perdiste, era el 7`.

B6.3 - El menú que no se rinde

Qué practicas: combinar `do-while` y `switch` para construir un menú que se repite.

Escribe un menú con `do-while`:

```
1) Saludar
2) Sumar dos numeros
3) Contar hasta 10
0) Salir
Opcion:
```

Requisitos:

- La opción 1 imprime un saludo.
- La opción 2 pide dos números y muestra la suma.
- La opción 3 cuenta del 1 al 10, uno por línea.
- Una opción que no existe muestra `Esa opcion no existe` y **vuelve** al menú.
- Solo el `0` termina el programa.
- Después de cada opción, el menú se muestra de nuevo.

¿Por qué `do-while` y no `while`? Porque el menú tiene que mostrarse al menos una vez antes de preguntar nada.

B6.4 - El validador insistente

Qué practicas: un `while` que no suelta al usuario hasta que el dato es válido, llevando la cuenta de los intentos.

Pide una nota de 1.0 a 7.0 y **no la sueltes** hasta que esté en rango. Cuenta cuántas veces se equivocó el usuario y díselo:

```
Nota (1.0 a 7.0): 9
Nota invalida. Otra vez: -2
Nota invalida. Otra vez: 5.5
Nota aceptada: 5.5 (te equivocaste 2 veces)
```

B6.5 - Desafío: dar vuelta un número

Qué practicas: usar un `while` en vez de un `for` cuando no sabes de antemano cuántas vueltas vas a dar.

Pide un entero positivo y muestra sus dígitos al revés, **sin** convertirlo a texto.

```
Numero: 5730
Al reves: 0375
Tiene 4 digitos
Suma de sus digitos: 15
```

Pista: `numero % 10` te da el último dígito, `numero / 10` te saca el último dígito. Repite mientras el número sea mayor que 0.

Cuando termines, sigue con el [paso 7](#).